

Arbre parfait	3
Définition : Arbre binaire parfait	3
Exemples	3
Propriétés.....	4
Numérotation d'un arbre parfait.....	7
Implémentation d'un Arbre parfait	8
Inversement	9
Tableau et arbre parfait	10
Implémentation	11
Primitives d'accès aux membres de la structure.....	12
Quelques Algorithmes d'arbres parfaits.....	14
Fonction initAP(a) → ArbreParfait.....	15
Fonction estVideAP(a) → Booléenne.....	15
Fonction estPleineAP(a)→Booléenne.....	15
Parcours d'un arbre parfait.....	17
File de priorité (Tas ou Heap)	22
But	22
Définition.....	22
Applications.....	23
Définition (tas ou file de priorité)	24
Tas et Tableau.....	26
Propriété d'un tas-Max	26
Implémentation	26
Opérations d'un tas	27
Trouver le maximum	28
Insertion dans un tas	29
Complexité d'une insertion dans un tas	34
Suppression du maximum : Suppression du nœud Racine	35
Complexité de la suppression	39

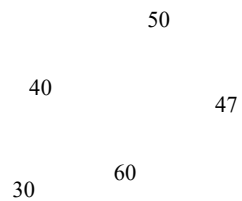
Application ; Heap Sort (Tris Par Tas).....	40
Exercice	45

ARBRE PARFAIT

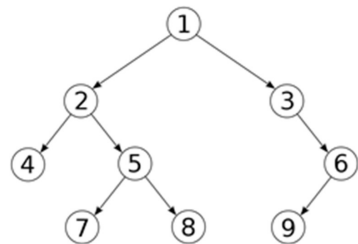
Définition : Arbre binaire parfait

Un arbre binaire de hauteur h est dit parfait si tous les niveaux de l'arbre 0, 1, ..., $h-1$ sont pleins et les éléments du dernier niveau h sont regroupés à gauche.

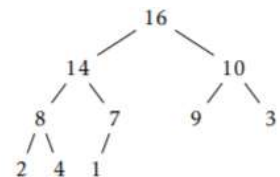
Exemples



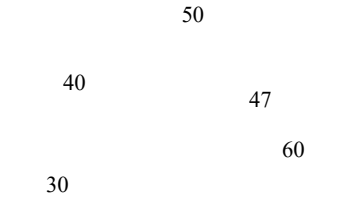
Arbre Parfait



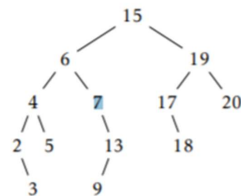
Ce n'est pas un arbre parfait



Arbre Parfait

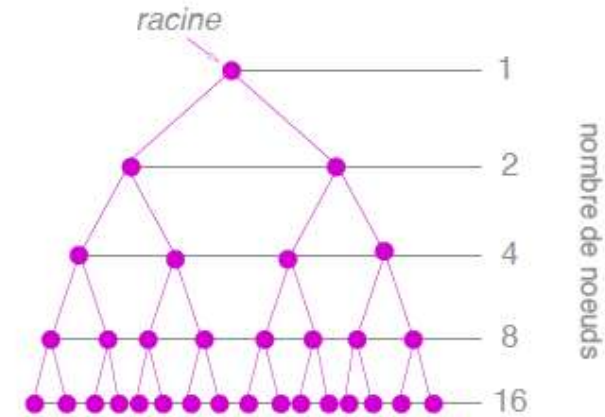


Ce n'est pas un Arbre Parfait



Ce n'est pas un arbre Parfait

Arbres binaires Complets



Arbre binaire complet de hauteur h

Propriétés

Propriété 1

Le nombre de nœuds dans un arbre parfait de hauteur h est $< 2^{h+1}$

Remarque

Dans un arbre parfait :

- A chaque niveau i de l'arbre, le nombre de nœuds $= 2^i$
- Au dernier niveau h , le nombre de nœuds $\leq 2^h$

Preuve :

Niveau 0 : nombre de nœuds $= 2^0$

Niveau 1 : nombre de nœuds $= 2^1$

Niveau i : nombre de nœuds $= 2^i$

Niveau h : nombre de nœuds $\leq 2^h$

Donc le nombre de nœuds au total est $\leq \sum_{i=0}^{h} 2^i = 2^{h+1} - 1 < 2^{h+1}$

Propriété 2 d'un arbre parfait

Un arbre parfait à n nœuds est de hauteur h vérifié la propriété suivante :

$$2^h \leq n < 2^{h+1}$$

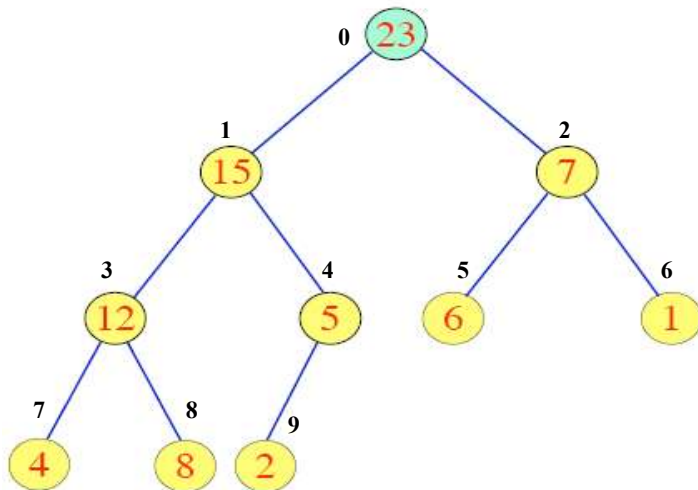
$$h \leq \log_2 n < h+1$$

Numérotation d'un arbre parfait

On numérote les nœuds de l'arbre parfait en effectuant un parcours « par niveaux » (on dit aussi en largeur). On constate qu'on associe à chaque nœud un entier de la manière suivante :

- ❖ La racine de l'arbre est étiquetée par 0
- ❖ Le fils gauche d'un nœud d'étiquette n , est étiquetée par $2n+1$
- ❖ Le fils droit d'un nœud d'étiquette n , est étiquetée $2n+2$
- ❖ Le père d'un nœud d'étiquette n (sauf le nœud racine), est étiquetée par $(n-1)/2$

Exemple



Remarque

Si un nœud avec une étiquette « n » existe dans A , alors tous les nœuds avec une étiquette inférieure à « n » existent aussi.

Implémentation d'un Arbre parfait

Un arbre parfait peut être représenté par un tableau « Tab ».

Numéro d'un nœud lors d'un parcours en largeur de l'arbre = Indice du tableau.

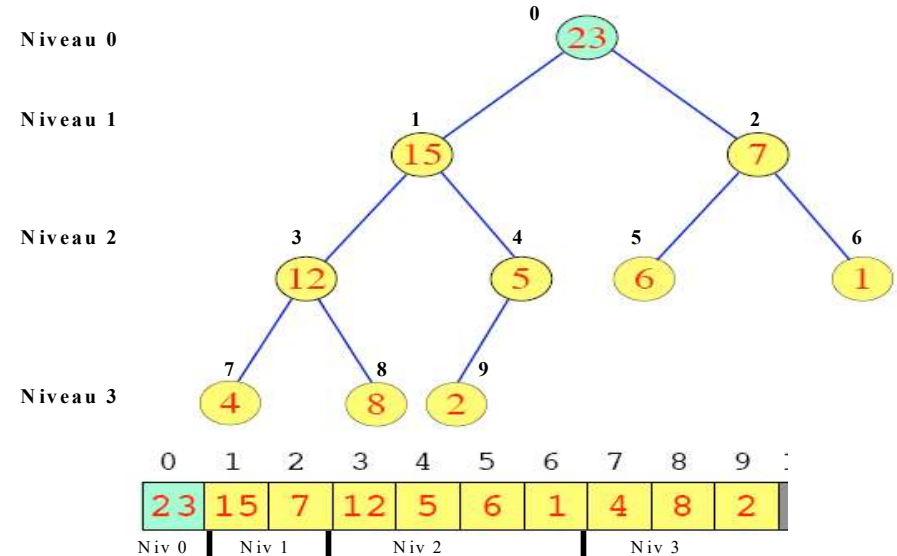
A la racine de l'arbre on associe l'indice 0

Tab[0] est la donnée associée à la racine de l'arbre

Pour un nœud d'étiquette n :

- ❖ L'indice du Parent du nœud n est $(n-1)/2$ (on suppose que n est différent de 0. Puisque le nœud racine n'a pas de père)
- ❖ L'indice du Fils gauche du nœud n est $2n+1$
- ❖ L'indice du Fils droit du nœud n est $2n+2$

Exemple



Inversement

On peut considérer que tout tableau $\text{Tab}[n]$ est la représentation d'un arbre parfait.

Dans ce cas, n n'est pas forcément égal à $2^{h+1}-1$.

Avec h la hauteur de l'arbre.

Cela signifie qu'au dernier niveau, il manque des feuilles.

Dès qu'il manque une feuille, il manque toutes les feuilles suivantes.

Dans le cas où n est égal à $2^{h+1}-1$, on dit aussi que l'arbre parfait est complet.

Tableau et arbre parfait

L'arbre parfait est représenté à l'aide d'un tableau $\text{Tab}[0..n[$ composé de n éléments (nœuds).

➤ Si i est l'indice d'un nœud de l'arbre, alors :

- Le nœud d'indice i est non vide correspond à la relation : $i < n$
- Le fils gauche de ce nœud a pour indice $2*i+1$,
- Le fils droit a pour indice $2*i+2$,
- $\text{Tab}[i]$ correspond à la donnée associée à ce nœud.

➤ La racine de l'arbre a pour indice **0**.

Implémentation

Langage algorithmique :

```
Type Vache TElement
Type ArbreParfait= Structure
    tailleMax : Entier // taille maximale
    taille : Entier    // taille courante
                    // Nbre de nœuds ds l'arbre parfait
                    // = nbre d'élts du tableau
    tab : Tableau // tab est un tableau de TElement
Fin Structure
```

Langage C

```
Typedef Vache. TElement;
Typedef struct [arbreparfait]{
    int taille ;
    int tailleMax;
    TElement *tab ;
} ArbreParfait ;
```

Primitives d'accès aux membres de la structure

//Retourne la taille associée à l'arbre Parfait

Fonction taille(a) → Entier

Debut

Renvoyer a.taille

Fin

//Retourne la taille max associée à l'arbre Parfait

Fonction tailleMax(a) → Entier

Debut

Renvoyer a.tailleMax

Fin

// Retourne la donnée associée au ième nœud de l'arbre parfait

Précondition : si le ième nœud existe (càd $0 \leq i < \text{taille}(a)$) :

Fonction iEmeElts(i, a) → TElement

Debut

Renvoyer a.tab[i]

Fin

Fonctions virtuelles exprimés sous forme d'arbre binaire

// Accès à la donnée d'un nœud donné « nd »

Précondition : si le nœud « nd » existe (càd $nd < \text{taille}(a)$)

Fonction donneeAP(nd, a) → TElement

Début

renvoyer iEmeElts(nd, a) // a.tab[nd]

Fin

// Retourne le fils gauche d'un nœud « nd » donné s'il existe

Fonction filsGaucheAP(nd [, a]) → entier

Début

Renvoyer $2*nd + 1$

Fin

// Retourne le fils droit d'un nœud « nd » donné s'il existe

Fonction filsDroitAP(nd [, a]) → entier

Début

Renvoyer $2*nd + 2$

Fin

// Retourne le père d'un nœud « nd » donné s'il existe

// Sauf pour le premier nœud (=racine) de l'arbre

Fonction pereAP(nd [, a]) → Entier

Début

Renvoyer $(nd-1)/2$

Fin

Quelques Algorithmes d'arbres parfaits

// Allocation mémoire de la structure d' arbre parfait

Fonction allocMemAP(tMax) → ArbreParfait

P.F tMax : Entier (E)

Début

a. tailleMax ← tMax

a.tab ← allocMemTab(nMax)

Renvoyer a

Fin

procedure allocMemAP(tMax, a)

P.F tMax : Entier (E)

a : ArbreParfait (E/S)

Début

a. tailleMax ← tMax

a.tab ← allocMemTab(nMax)

Fin

//libération mémoire de la structure d'arbre parfait

Procédure libMemAP(a)

P.F a : ArbreParfait (E/S)

Début

libMemTab(a.tab)

Fin

Fonction libMemAP(a)→ ArbreParfait

P.F a : ArbreParfait (E)

Début

libMemTab(a.tab)

Renvoyer a

Fin

// Initialisation d'un arbre Parfait

Fonction initAP(a) → ArbreParfait

```
P.F a : ArbreParfait (E)
Début
    a.taille ← 0
    Renvoyer a
Fin
```

// Teste si un arbre Parfait s'il est vide ou pas

Fonction estVideAP(a) → Booléenne

```
P.F a : ArbreParfait (E)
Début
    Renvoyer taille(a) = 0
Fin
```

// Teste si un arbre Parfait s'il est plein ou pas

Fonction estPleineAP(a) → Booléenne

```
P.F a : ArbreParfait (E)
Début
    Renvoyer taille(a) >= tailleMax(a)
Fin
```

// Teste la non existence d'un nœud dans un Arbre Parfait

Fonction estVideNoeud(nd, a) → booléenne

```
P.F nd : Entier (E) a : ArbreParfait (E)
Début
    Renvoyer nd >= taille(a)
Fin
```

// Teste l'existence d'un nœud dans un Arbre Parfait

Fonction estExistNoeud(nd, a) → booléenne

```
P.F nd : Entier (E) a : ArbreParfait (E)
Début
    Renvoyer nd < taille(a)
Fin
```


Parcours d'un arbre parfait

Parcours Préfixe = R G D

Procédure ParcoursPrefixe(racine, a)

Paramètres formels

racine : entier (E)

A : arbreParfait (E)

début

si **estExistNoeud(racine, a)** alors

 traiter(donneeAP(racine, a))

 ParcoursPrefixe(filsGaucheAP(racine, a), a)

 ParcoursPrefixe(filsDroitAP(racine, a), a)

Finsi

Fin

Parcours infix = G R D

Procédure ParcoursInfixe(racine, a)

Paramètres formels

Racine : entier (E)

A : arbreParfait (E)

début

si **estExistNoeud(racine, a)** alors

 ParcoursInfixe(filsGaucheAP(racine, a), a)

 traiter(donneeAP(racine, a))

 ParcoursInfixe(filsDroitAP(racine, a), a)

Finsi

Fin

Parcours infixe = G D R

Procédure **ParcoursPostfixe**(racine, a)

Paramètres formels

racine : entier (E)

a : ArbreParfait (E)

début

si **estExistNoeud**(racine, a) alors

ParcoursPostfixe(filsGaucheAP(racine, a), a)

ParcoursPostfixe(filsDroitAP(racine, a), a)

traiter(donneeAP(racine, a))

Finsi

Fin

Parcours en Largeur



Parcours du tableau

void **parcoursLargeur**([int racine, [ArbreParfait a]){

Paramètres formels

racine : entier (E)

a : ArbreParfait (E)

Début

nbNoeuds $\leftarrow$ taille(a)

pour nd variant de 0 à (nbNoeuds-1) faire

traiter(donneeAP(nd, a))

finPour

Fin

Parcours en Largeur

Affichage niveau / niveau de l'arbre Parfait

```
void parcoursLargeur(int racine, ArbreParfait a){  
  
    voir TD  
  
}
```

FILE DE PRIORITÉ (TAS OU HEAP)

But

Traiter un ensemble d'éléments e appartenant un ensemble E muni d'une relation d'ordre ($\leq$) selon une clé appelée leur priorité $p(e)$

On distingue deux types de files de priorité :

- **Priorité maximale**
- **Priorité minimale**

Remarque

- Nous présentons la première mais la seconde est identique dans le principe (à la fonction de comparaison près).
- **Convention :** on confondra chaque élément e avec sa priorité $p(e)$. C'est à dire, on considère l'élément e est égal à sa priorité $p(e)$.

Définition

Une file de priorité est un type abstrait de données opérant sur un ensemble ordonné, et muni des opérations suivantes :

- Trouver le maximum  sommetF(f)
- Insérer un élément  enfiler(elt, f)
- Retirer le maximum  defiler(f)

Applications

On retrouvera donc un tas dans tous les algorithmes qui demandent de gérer des priorités :

- Algorithme de **Huffman (compression)**
- Algorithmes de graphes (**Prim, Dijkstra**)

Application « évidente » : tri par tas (*heapsort*) :

D'autres applications

- Ordonnancement des tâches d'un système d'exploitation.

Par exemple, la file des tâches à exécuter peut-être ordonnée suivant la durée des tâches : on donnera la priorité aux tâches courtes.

- Gestion des ressources dans un système d'exploitation.
- Techniques de simulation basée sur les événements.

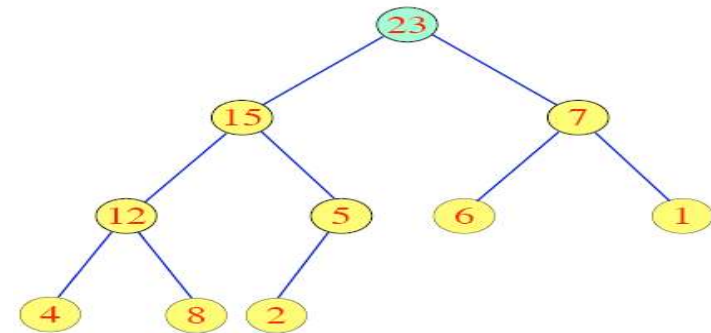
Par exemple, pour simuler un système d'événements (ligne de bus, ascenseurs, etc) en classe les événements suivant leur date.

Définition (tas ou file de priorité)

Un tas est un arbre binaire parfait tel que le contenu de chaque nœud soit supérieur ou égal à celui de ses fils (tous ses descendants) : c'est la condition de tas.

De plus, chaque niveau de l'arbre doit être rempli de gauche à droite.

Exemple



Autrement dit : Un tas est un arbre binaire parfait dans lequel tout nœud non terminal possède la propriété suivante :

$$\text{Donnee(Nœud)} \geq \text{Max(donnee(filsGauche(nœud)) , donnee(filsDroit(nœud)))}$$

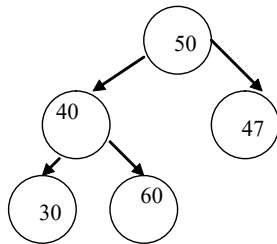
C'est à dire la donnée associée au nœud père est supérieure ou égale aux données associées à ses deux fils.

Dans le cas où le **nœud n'a qu'un seul fils** (c'est nécessairement un fils gauche) ; la propriété se réduit à :

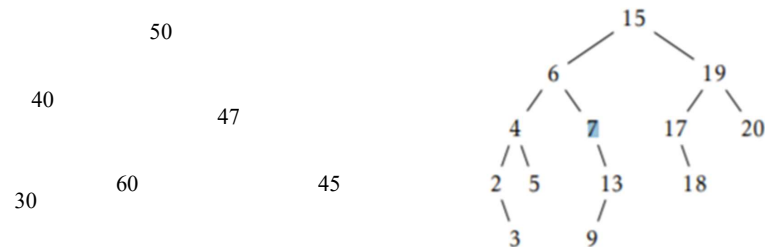
$$\text{donnee(Nœud)} \geq \text{donnee(filsGauche(nœud))}$$

Exemples

Exemple d'arbre parfait qui n'est pas un Tas.



Exemple d'arbre binaire qui n'est pas parfait → n'est pas un tas



Tas et Tableau

Soit $\text{Tab}[n]$ la représentation d'un tas composé de n éléments.

Si i est l'indice d'un nœud alors

$$\text{Tab}[i] \geq \text{Max}(\text{Tab}[2*i+1], \text{Tab}[2*i+2]) \text{ et } (2*i+2 < n)$$

$$\text{Où } \text{Tab}[i] \geq \text{Tab}[2*i+1] \text{ et } (2*i+1 < n)$$

Propriété d'un tas-Max

Dans un tas-Max, la valeur associée à la racine (donc le premier élément du tableau) est le maximum de tous les éléments : $\text{Tab}[0] \geq \text{Tab}[1..(n-1)]$

Implémentation

Structure Tas $\Leftrightarrow$ Structure Arbre Parfait

```

Typedef .... TElement ;

Typedef struct tas{

    int tailleMax ;

    int taille ;

    TElement *tab ;

} Tas ;
    
```

typedef ArbreParfait Tas ;

Opérations d'un tas

Précondition non estVideTas(t)

- Trouver le maximum  sommetF(f)

Précondition non estPleinTas(t)

- Insérer un élément  enfiler(elt, f)

Précondition non estVideTas(t)

- Retirer le maximum  defiler(f)

Trouver le maximum

Précondition : non estVideTas(t)

➤ Le maximum est le contenu du nœud racine (=0)

Fonction maxTas (t) → Telement

P.F t : Tas (Paramètre en Entré)

Début

 Renvoyer donneeAP(0, t) // ou t.tab[0]

Fin

Insertion dans un tas

Principe

L'insertion d'un nouvel élément e se fait en deux temps :

- Placer le nouveau nœud contenant l'élément e à la première place libre en respectant la contrainte d'arbre parfait.
- Permuter avec le nœud parent jusqu'à l'obtention d'un tas ; c'est à dire on échange la donnée de la nouvelle feuille avec la donnée de son père, puis la donnée de celui-ci avec la donnée de son père et ainsi de suite jusqu'à ce que la propriété $\text{donnee}(\text{pere}(e)) \geq \text{donnee}(e)$ est satisfaite.

Exemple

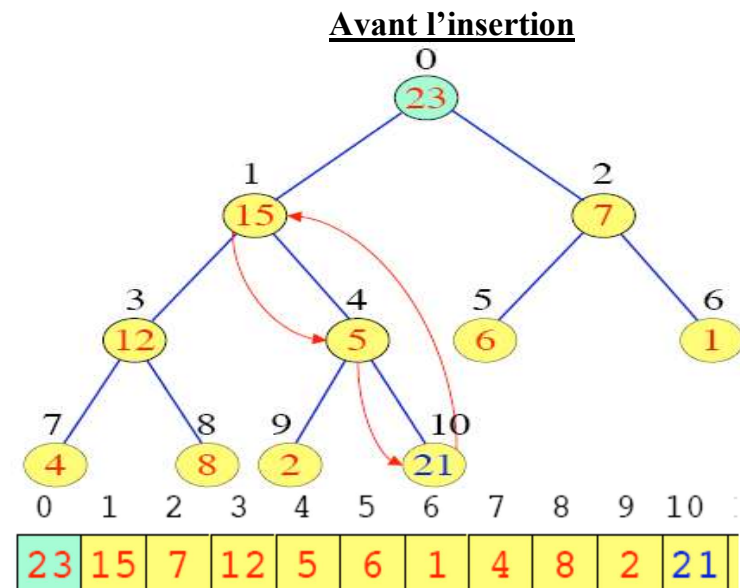
Insertion de la valeur 21.

- On place la valeur 21 à droite de la dernière feuille introduite, c'est-à-dire dans la case d'indice 10.
- Permutation du nouveau nœud avec le nœud parent jusqu'à l'obtention d'un tas c'est à dire faire remonter la valeur du fils en père afin de respecter la contrainte tas.
 - ❖ On compare 21, la nouvelle donnée insérée, avec la donnée contenue dans le nœud père,

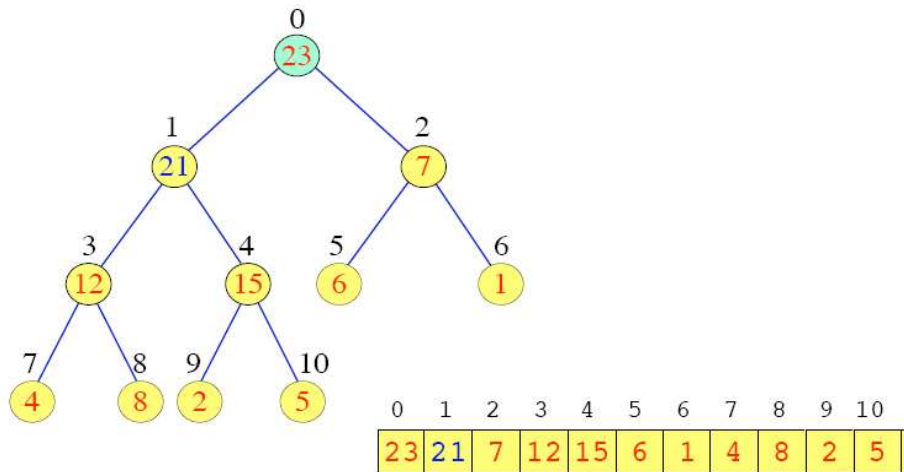
Puisque 21 est plus grand que 5, on échange le 21 et le 5
 - ❖ On compare 21, avec la donnée contenue dans le nœud père,

Puisque 21 est plus grand que 15, on échange le 21 et le 15
 - ❖ On compare 21, avec la donnée contenue dans le nœud père,

Puisque 21 est plus petit que 23, l'insertion est terminée.



Après insertion de 21



Algorithme d'insertion : Version itérative

Précondition : non estPleinTas(t)

Procédure inserTas(e, t)

P.F e : Telement (E) t : Tas (E/S)

Début

n ← taille(t) //n=place de l'insertion du nouveau élément

si n = 0 alors // (eq au estvideTas) → On effectue l'insertion

t.tab[n] ← e

t.taille ← taille(t) + 1

sinon // on cherche le nœud d'insertion

p ← pere(n, t)

dp ← donneeAP(p, t) // t.tab[p]

Tantque (n > 0 et dp < e) faire // 0 = nœud racine

// on déplace le contenu du père vers le fils

t.tab[n] ← dp

// le fils deviendra le père

n ← p

p ← pere(n)

dp ← donneeAP(p, t)

fintq

// n = 0 ou e ≤ dp

// On effectue l'insertion

t.tab[n] ← e

t.taille ← taille(t) + 1

fsi

Fin

Algorithme d'insertion : version récursive

Raisonnement

Soit $n = \text{taille}(t)$ // le nœud éventuelle d'insertion

➤ **Cas où $n=0$ (càd soit le tas est vide soit l'elt > aux elts du tas)**

//on réalise l'insertion

$t.\text{Tab}[0] \leftarrow e$

Incrémenter la taille du tas

➤ **Cas $n>0$ // n est aussi la place d'insertion**

$p \leftarrow \text{pere}(n)$

$dp \leftarrow \text{donneeAP}(p, t)$

on distingue deux cas :

* $e \leq dp$: // on effectue l'insertion

$t.\text{tab}[n] \leftarrow e$

$t.\text{taille} \leftarrow \text{taille}(t) + 1$

* $e > dp$: //on déplace le contenu du père vers le fils

$t.\text{tab}[n] \leftarrow dp$

// le fils deviendra le père

$n \leftarrow p$

...

Faire l'appel récursif

Complément en TD

Complexité d'une insertion dans un tas

Lorsqu'on insère un élément dans un tas, ce dernier est inséré initialement à la première place libre en respectant la contrainte d'arbre parfait, c'est-à-dire dans le niveau de profondeur h , c'est-à-dire à la dernière place dans le tableau.

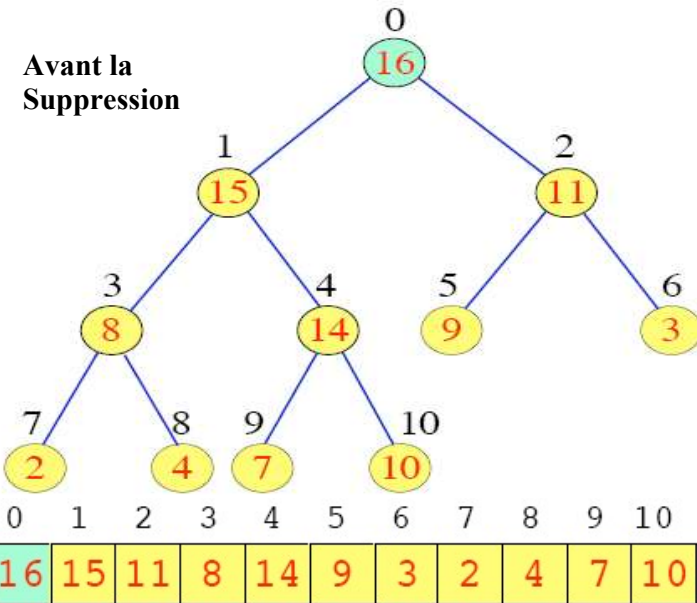
Donc, on effectue **au plus** h comparaisons et échanges d'éléments.

Suppression du maximum : Suppression du nœud Racine

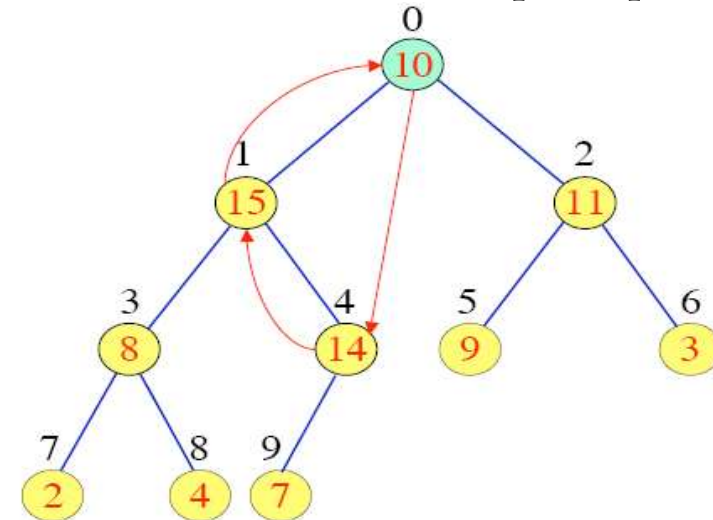
Principe

- Placer le dernier nœud à la racine
- Supprimer le dernier nœud
- Echanger avec le plus grand fils jusqu'à l'obtention d'un tas

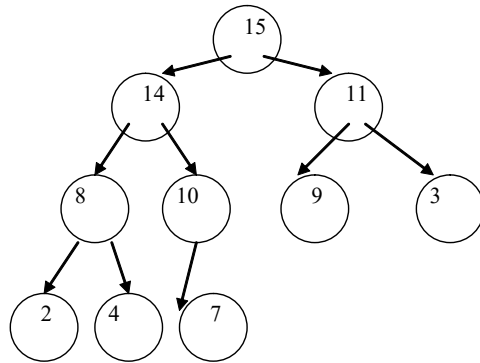
Exemple :



Faire descendre la valeur de la racine pour respecter l'ordre



Après suppression



15	14	11	8	10	9	3	2	4	7
----	----	----	---	----	---	---	---	---	---

Algorithme de la suppression du maximum

Precondition : non estVideTas(t)

```

Procédure SuppTas(t)  P.F t : Tas (E/S)
Début
    r ← 0  //nœud racine
    // Placer le dernier nœud à la racine & suppression du dernier nœud
    dn ← taille(t)-1
    t.tab[r] ← donneeAP(dn, t)
    // Supprimer le dernier nœud = décrémenter la taille du tas
    t.taille ← dn
    si non estVideTas(tas) alors
        //Echanger le nœud courant (au départ le nœud racine) avec le
        // plus grand fils jusqu'à l'obtention d'un tas
        e ← donneeAP(r, t)
        échange ← vrai
        fg ← filsGaucheAP(r, t)
        tantque échange et estExistNoeud(fg, t)) faire
            fd ← filsDroitAP(r, t)
            fmax ← fg
            // Vérification de l'existence du fils droit & rech du fMax
            si estExistNoeud(fd,t) et
                donneeAP(fd,t)>donneeAP(fg,t)
            alors
                fmax ← fd
            finsi
            si e >= donneeAP(fmax, t) alors
                échange ← faux
            sinon
                t.tab[r] ← donneeAP(fmax, t)
                r ← fmax
                fg ← filsGaucheAP(r, t)
            finsi
        fintq
        //Placer la donnée de la feuille à la bonne
        place
        t.tab[r] ← e
    FinSi
Fin
    
```

Voir TD

Complexité de la suppression

La complexité est au pire de l'ordre de la hauteur de l'arbre binaire parfait, puisque pour chaque niveau de l'arbre, on fait un nombre d'opérations (aux plus deux comparaisons et quelques affectations).

Problème

On souhaite trier un tableau `tab` composé de n éléments.

Principe

On **construit un tas** en ajoutant successivement au Tas vide les éléments : `Tab[0], ..., Tab[n-1]`

On répète ensuite les opérations suivantes :

- Prendre le maximum du Tas
- Supprimer le maximum du Tas
- Le mettre à droite du tableau

Libérer la mémoire occupée par le tas

Algorithme : Heap Sort**Procédure TriParTas(n, Tab)**

Paramètres formels

n : entier (E) tab : type Tableau (E/S)

Début

// Initialisation et allocation Mémoire

tas ← allocMemArbre(n, tas) //appel fct

tas ← initArbre(tas) // appel fct

//Construction du tas

Pour i variant de 1 à n faire

InsertTas(tab[i], tas) // appel proc

Fpour

//Retirer les éléments un à un et les placer dans le tableau en commençant par la fin

pour i variant de n à 1 faire

e ← MaxTas(tas) // appel fct

suppTas(tas) // appel proc

tab[i] ← e

fpour

//Libération Mémoire

Tas ← libMemAP(tas) // appel fct

Fin**Variables Intermédiaires**

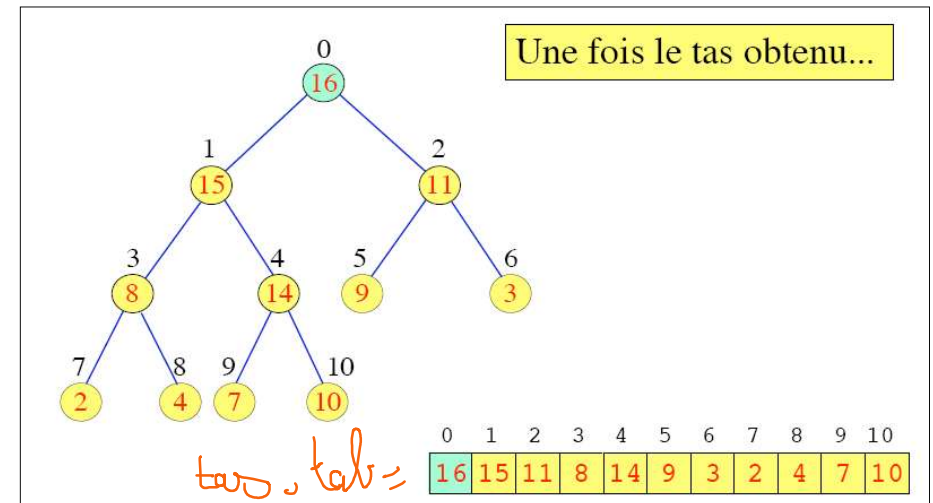
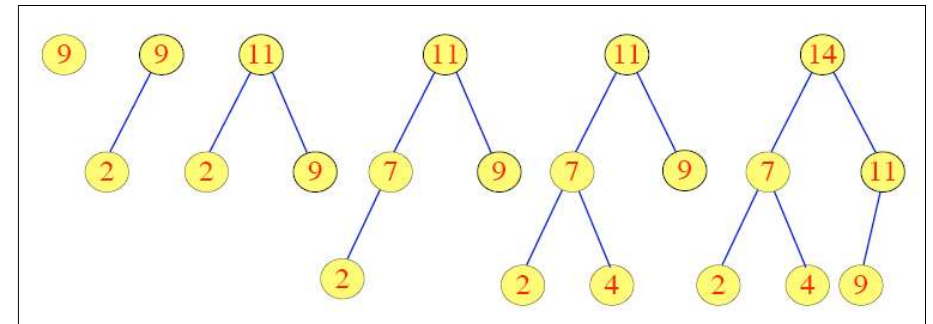
tas : type Tas e : type Telement

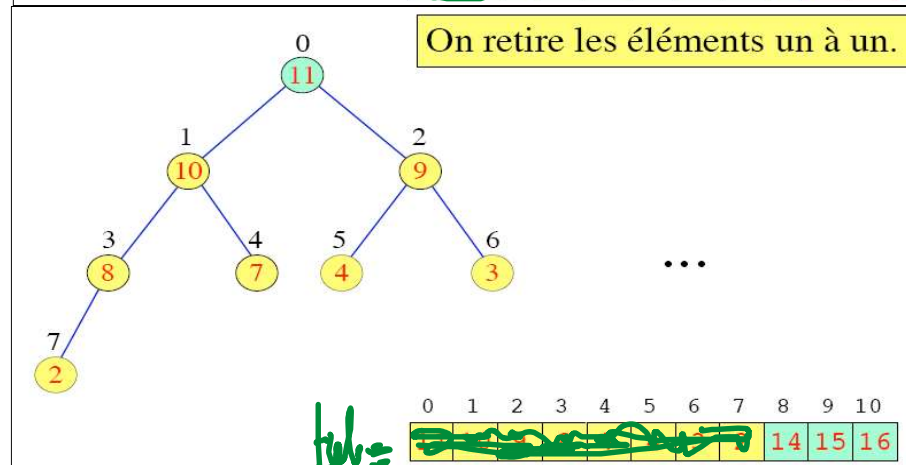
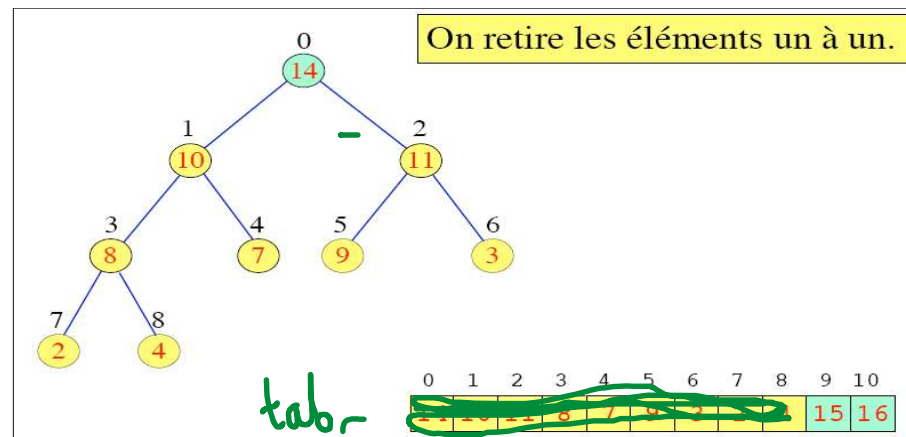
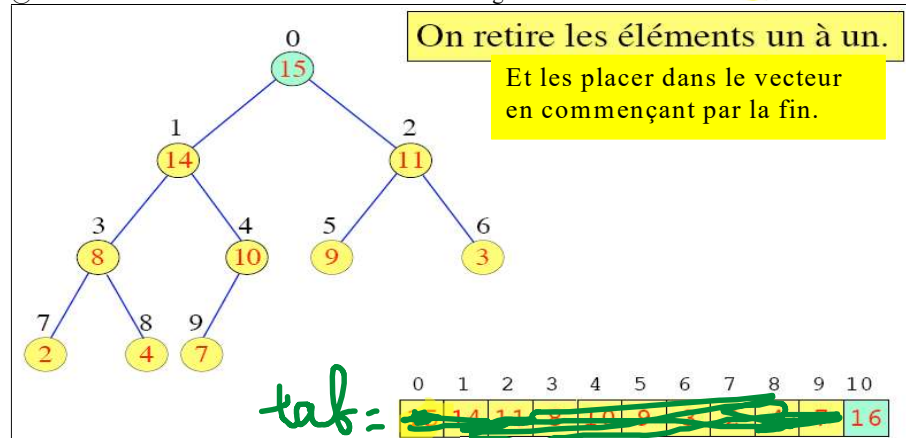
Exemple

Afin de comprendre l'algorithme HeapSort, nous allons raisonner sur l'arbre alors que les actions seront effectuées sur sa représentation Tab[n]. L'arbre n'est ici qu'un objet virtuel nécessaire à la compréhension des actions faites sur le tableau Tab.

Soit Tab[8]=[9, 2, 11, 7, 4, 14, 3, 16, 8, 10, 15] le tableau à trié

On construit un arbre on ajoutant un à un les éléments :





Rappel Propriété d'un arbre parfait

Un arbre parfait à n nœuds est de hauteur h vérifié la propriété suivante :

$$2^h \leq n < 2^{h+1}$$

$$h \leq \log_2 n < h+1$$

Conséquence :

Les opérations (insertions et suppressions) ne parcouraient qu'une branche de l'arbre parfait, elle serait de complexité $\leq h = \Theta(\log_2 n)$

Complexité du tri par tas dans le pire des cas

- n opérations insérer coûtant chacune $O(\log_2 n)$
- n opérations supprimer coûtant chacune $O(\log_2 n)$

➡ Complexité du tri est de l'ordre $n \log_2 n$

Remarque

Ce tri interne nécessite une mémoire auxiliaire (tas) qui est de l'ordre de n .

Exercice

Ecrire une fonction booléenne qui vérifie si un tableau `tab` composé de `n` éléments peut être un tas.

Fonction `estTas(n, tab) → Booléenne`

P.F

`n` : entier = nbre d'éléments présents dans la tableau

`tab` : tableau (on suppose que le premier élément à pour indice 0)

/*****/

➤ **Version 1** : raisonnement de la droite vers la gauche : on compare la donnée associée à chaque nœud avec la donnée associée à son nœud père.

➤ **Version 2** : Raisonnement de la gauche vers la droite : on suppose que le tas est transformé en arbre parfait. On n'effectue aucune transformation par contre on raisonne d'une façon virtuelle.